

Currently, Requests only tries to get and automatically convert the response data into a Python object if the response *mimetype* is *application/json*.

Parameters

Let's make a request with a couple of URL parameters. We can always construct the request URL manually, like this:

```
>>> url = "http://example.com/param?arga=one&argb=2"
>>> r = requests.get(url)
```

This works, but there's a simpler way. We can create an argument payload (basically a dictionary), as shown below:

```
>>> args = {"arga": "one", "argb": 2}
```

And then make a request as follows:

```
>>> r = requests.get("http://example.com/param", params=args)
```

If you want to verify the URL that the request was made to, print `r.url` (where `r` is the response object), and you'll see that the URL was correctly constructed:

```
>>> r.url
'http://example.com/param?arga=one&argb=2'
```

If you want to make a POST request to the same endpoint with the same data, you can use the following code:

```
>>> r = requests.post("http://example.com/param", data=args)
```

Note that with POST, PUT, DELETE and the rest, you use the data argument. As the data is a dictionary, Requests will form-encode the data and send it. To get past this, you can simply pre-encode the data into a string and pass it. So to send a JSON request body (such as when you're making an API call), you can simply do the following:

```
>>> r = requests.post("http://example.com/param", data=json.
dumps(args))
```

Let's upload a file. Requests makes it easy to do Multipart-Encoded (basically, chunked) uploads - all you have to do is provide a file-like object. Something like what follows should do nicely:

```
>>> url = 'http://example.com/file-upload'
>>> files = {"file": ("report.xls", open("report.xls", "rb"))}
>>> r = requests.post(url, files=files)
```

I should explain the files dictionary a bit. The keys to the dictionary are the names of the file-input form fields that you would create in a HTML form. The value tuple

has two values inside it - the first one is the file name you want the server to see, and the second is the file-like object that Requests reads the data from.

Easy enough? Let's try sending custom headers.

Let's suppose we have an API endpoint that can accept data serialised as both JSON and XML. Then, we would need to specify what format we're sending by populating the Content-Type request header with the appropriate mimetype.

Now, let's send some JSON data to the aforementioned endpoint:

```
>>> import simplejson as json
>>> url = "https://example.com/api_endpoint"
>>> payload = {"pi": 3.14159, "e": 2.71828}
>>> headers = {"Content-Type": "application/json"}
>>> r = requests.post(url, data=json.dumps(payload),
headers=headers)
```

That's it! All you need to do is populate the headers argument with a dictionary containing your headers. Requests will automatically turn it into a case-insensitive dictionary, so you don't have to go digging for *CaseInsensitiveDict* yourself.

Let's try something different now...something tasty!

Cookies

You might need to deal with HTTP cookies in your applications. 'Requests' makes it pretty easy to do so. Every response object has a cookies property that holds all the cookies the server passed down to you. Let's see what it looks like:

```
>>> r.cookies
<<class 'requests.cookies.RequestsCookieJar'>[Cookie(version=0, name='NID', value='67...
```

To access individual cookies, run the following commands:

```
>>> r.cookies.get("NID")
'67=BbB9rqYqjGgH_
>>> r.cookies["NID"]
'67=BbB9rqYqjGgH_
```

Right, so it behaves like a dictionary.

Sending cookies is equally easy. All you need to do is load up the cookies argument to the request method with a dictionary, as shown below:

```
>>> url = "http://example.com/cookies"
>>> cookies = {"the_answer_to_life_the_uni
everything": 42}
>>> r = requests.get(url, cookies=cookies)
```

Let's try something a little more mundane now.